→ Embedding Table :-

→ input token

$I = [2, 0, 2]$

↓ embedding matrix

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Position embedding (P)

$$P = \begin{bmatrix} 0.1 & 0.2 & 0.3 & 0.4 \\ 0.5 & 0.6 & 0.7 & 0.8 \\ 0.9 & 1.0 & 1.1 & 1.2 \end{bmatrix}$$

$\oplus$

$$H = X + P$$

Addition Gradient

$$\frac{\partial L}{\partial X_{i,d}} = \sum_{i=0}^{N-1} \sum_{k=0}^{D} \frac{\partial L}{\partial H_{i,k}} * \frac{\partial H_{i,k}}{\partial X_{i,d}}$$

since $\frac{\partial H_{i,k}}{\partial X_{i,d}} = \partial_{i,i} \partial_{k,d}$

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial H} \text{ and } \frac{\partial L}{\partial P} = \frac{\partial L}{\partial H}$$

Backward pass:

$$X_{i,d} = \sum_{v=0}^{V-1} \delta_{I,v} W_{v,d}$$

↓ compute derivate with respect $\text{Emd}(W_{j,d})$

$$\frac{\partial X}{\partial W_{j,d}} = \delta_{I_i,j}$$

→

$$dX = dH, \; dP = dH$$

→ Embedding backward :-

$$dW_j = \sum_{i : I_i = j} dX_i$$

→ Why atomic_add?

worker 1 :- $w_2 += G_1$

worker 02 :- $w_2 += G_2$

⟷ without atomic

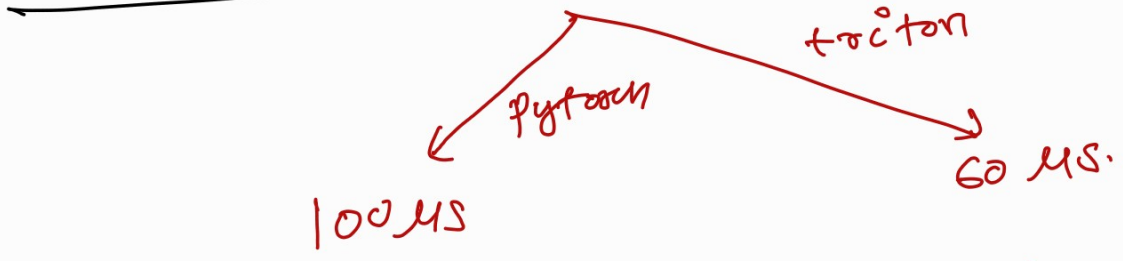
$w_2 \leftarrow w_2 + G_1$

$w_2 \leftarrow w_2 + G_2$

⟷ with atomic

Ex°:- $V = 50K, D = 1024, N = 2048,$ FP16

operation	Memory
Forward NXD	4.19MB
Backward VXD	102.4MB.



→ Forward :-

$X = WF$

Backward:-

$dw = scatter_add(G, I)$

V25, D=3, M=4, $W \in R^{VXD}$, $I \in \{0,1,2,\dots V-1\}^M$

where $I = [2,0,2,4]^T$

→ The forward pass as matrix:-

$$P_{m,v} = \delta_{I_{m,v}} = \begin{cases} 1 & \text{if } I_m = v \\ 0 & \text{if } I_m \neq v \end{cases}$$

$I = [2, 0, 2, 4]^T$, $P = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$

→ The forward pass :-

$$X = PW \in R^{M \times D}$$

$$X = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \\ 13 & 14 & 15 \end{bmatrix} = \begin{bmatrix} 7 & 8 & 9 \\ 1 & 2 & 3 \\ 7 & 8 & 9 \\ 13 & 14 & 15 \end{bmatrix}$$

→ The Backward pass as the adjoint (Transpose) operator :-

→ $L \in$ Loss, $G \in$ Gradient

$$G = \nabla_X L \in R^{M \times D} \longrightarrow \langle A, B \rangle = Tr(A^T B) :-$$

↳

$$\partial L = Tr(G^T dX)$$

↓ substitute the total differential $dX = P dW$

$$\partial L = Tr(G^T (P dW))$$

$$= Tr((P^T G)^T dW)$$

$$= \langle P^T G, dW \rangle \quad \because \nabla_W L = \frac{\partial L}{\partial W}$$

$$\partial W = P^T G$$

→ $W = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \\ 13 & 14 & 15 \end{bmatrix}$, $I = [2, 0, 2, 4]$

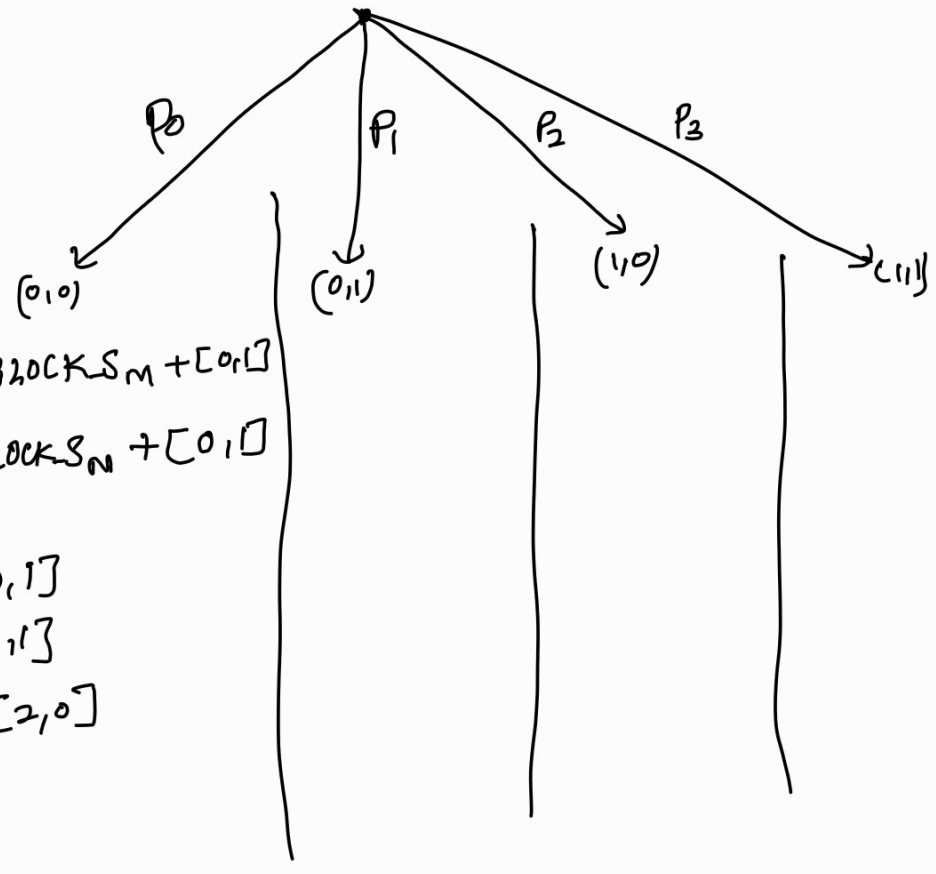
$X = W[I] \rightarrow \begin{bmatrix} 7 & 8 & 9 \\ 1 & 2 & 3 \\ 7 & 8 & 9 \\ 13 & 14 & 15 \end{bmatrix}$

→ How Grid ton maps this matrix to GPU Block.

$M = 4, \; N = 4, \quad BLOCK_SIZE_M = 2$

$BLOCK_SIZE_N = 2$

→ $\frac{4}{2} * \frac{4}{2} = 2 \times 2 = 4$ ← Blocks.



→ $offsets_m = pid_m * BLOCK_S_M + [0,1]$

→ $offsets_n = pid_n * BLOCK_S_N + [0,1]$

Ex:- $P_0 = (0,0)$

$offsets_m = [0,1]$

$offset_n = [0,1]$

$I[0:2] = [2,0]$

→ $G_2 = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{bmatrix}$

$dW \in R^{5 \times 3}$

$dW_0 = [4, 5, 6]$

$dW_2 = [\text{——}]$

$dW_4 = [\text{——}]$

Method 01:-

forward kernal

```
@triton.jit
def embedding_forward_kernel(
    embeddings_ptr,
    indices_ptr,
    output_ptr,
    n_elements,
    embedding_dim: tl.constexpr,
    BLOCK_SIZE_M: tl.constexpr, # How many embeds to load at a time
    BLOCK_SIZE_N: tl.constexpr,  # Block along the embed vector we want to load
    DTYPE_FLAG: tl.constexpr
):

    pid_m = tl.program_id(0)
    pid_n = tl.program_id(1)

    ### Cast Pointers ###
    indices_ptr = tl.cast(indices_ptr, tl.pointer_type(tl.int32))
    embeddings_ptr = tl.cast(embeddings_ptr, tl.pointer_type(tl.float32 if DTYPE_FLAG == 0 else (tl.float16 if DTYPE_FLAG == 1 else tl.bfloat16)))
    output_ptr = tl.cast(output_ptr, tl.pointer_type(tl.float32 if DTYPE_FLAG == 0 else (tl.float16 if DTYPE_FLAG == 1 else tl.bfloat16)))

    ### Get the indexes of what embeddings we are loading ###
    start_m = pid_m * BLOCK_SIZE_M
    offsets_m = start_m + tl.arange(0, BLOCK_SIZE_M)
    mask_m = offsets_m < n_elements
    indices = tl.load(indices_ptr + offsets_m, mask=mask_m, other=0)

    ### Get the indexes of what part of those embeddings we are loading ###
    start_n = pid_n * BLOCK_SIZE_N
    offsets_n = start_n + tl.arange(0, BLOCK_SIZE_N)
    mask_n = offsets_n < embedding_dim

    ### Get the combined mask where our indexing is valid ###
    combined_mask = mask_m[:, None] & mask_n[None, :]

    ### Get the offsets now (BLOCK_SIZE_M, 1) + (1 x BLOCK_SIZE_N)###
    ### each embedding has embed_dim number of values, so we advance by that ###
    ### much to get the correct starting pointer ###
    embedding_offsets = indices[:, None] * embedding_dim + offsets_n[None, :]
    embeddings = tl.load(embeddings_ptr + embedding_offsets,
                         mask=combined_mask,
                         other=0.0)

    ### This is just a copy op, we grabbed from our embedding matrix and place into our output matrix ###
    output_offsets = offsets_m[:, None] * embedding_dim + offsets_n[None, :]
    tl.store(output_ptr + output_offsets, embeddings, mask=combined_mask)
```

Backward kernel

```
@triton.jit
def embedding_backward_kernel(
    grad_output_ptr,
    grad_weight_ptr,
    indices_ptr,
    n_elements,
    embedding_dim: tl.constexpr,
    BLOCK_SIZE_M: tl.constexpr,
    BLOCK_SIZE_N: tl.constexpr,
    DTYPE_FLAG: tl.constexpr, # 0 for float32, 1 for float16, 2 for bfloat16
):

    pid_m = tl.program_id(0)
    pid_n = tl.program_id(1)

    ### Cast Pointers ###
    indices_ptr = tl.cast(indices_ptr, tl.pointer_type(tl.int32))
    grad_weight_ptr = tl.cast(grad_weight_ptr, tl.pointer_type(tl.float32 if DTYPE_FLAG == 0 else (tl.float16 if DTYPE_FLAG == 1 else tl.bfloat16)))
    grad_output_ptr = tl.cast(grad_output_ptr, tl.pointer_type(tl.float32 if DTYPE_FLAG == 0 else (tl.float16 if DTYPE_FLAG == 1 else tl.bfloat16)))

    ### Get the indexes of what embeddings we are loading ###
    start_m = pid_m * BLOCK_SIZE_M
    offsets_m = start_m + tl.arange(0, BLOCK_SIZE_M)
    mask_m = offsets_m < n_elements
    indices = tl.load(indices_ptr + offsets_m, mask=mask_m, other=0)

    ### Get the indexes of what part of those embeddings we are loading ###
    start_n = pid_n * BLOCK_SIZE_N
    offsets_n = start_n + tl.arange(0, BLOCK_SIZE_N)
    mask_n = offsets_n < embedding_dim

    ### Get the grad cooresponding to this block ###
    grad_output = tl.load(
        grad_output_ptr + offsets_m[:, None] * embedding_dim + offsets_n[None, :],
        mask=mask_m[:, None] & mask_n[None, :],
        other=0.0
    )

    ### Now this is an accumulate op. We want to add the grad to our embeddings grad ###
    ### at their cooresponding indices ###
    grad_weight_offsets = indices[:, None] * embedding_dim + offsets_n[None, :]

    ### Atomic add to handle race conditions ###
    tl.atomic_add(
        grad_weight_ptr + grad_weight_offsets,
        grad_output,
        mask=mask_m[:, None] & mask_n[None, :],
    )
```

→ Method 02 :—

← Fused_embedding_forward

```
def fused_embedding_forward(embeddings, indices, use_dlpack=True):

    ### Only support int32 indexes ###
    indices = indices.astype("int32", copy=False)

    ### Bounds checks force a GPU sync every call, so they are opt in ###
    if CHECK_INDICES:
        assert (indices >= 0).all(), f"Negative indices found: {indices.min()}"
        assert (indices < embeddings.shape[0]).all(), f"Index out of bounds: max={indices.max()}, vocab={embeddings.shape[0]}"

    if use_dlpack:

        ### Get the original shape ###
        original_indices_shape = indices.shape

        ### Flatten ###
        indices = indices.reshape(-1)

        ### Convert to torch ###
        embeddings = torch.utils.dlpack.from_dlpack(embeddings)
        indices = torch.utils.dlpack.from_dlpack(indices)

        ### Check Contiguous ###
        if not embeddings.is_contiguous():
            embeddings = embeddings.contiguous()
        if not indices.is_contiguous():
            indices = indices.contiguous()

        ### Create an empty output to copy embeds into ###
        output = torch.empty(
            indices.shape[0], # Number of tokens in total
            embeddings.shape[1], # Embed dim per token
            device=indices.device,
            dtype=embeddings.dtype
        )

        ### Get our dimensions ###
        n_elements = indices.shape[0]
        embed_dim = embeddings.shape[1]

        ### Vocab size is typically much larger than the embed_dim so lets set our ###
        ### block size based on the embed dim! ###
        BLOCK_SIZE_M = triton.next_power_of_2(min(128, embed_dim))
        BLOCK_SIZE_N = triton.next_power_of_2(min(128, embed_dim))

        grid = (
            triton.cdiv(n_elements, BLOCK_SIZE_M),
            triton.cdiv(embed_dim, BLOCK_SIZE_N)
        )

        embedding_forward_kernel[grid](
            embeddings_ptr=embeddings,
            indices_ptr=indices,
            output_ptr=output,
            n_elements=n_elements,
            embedding_dim=embed_dim,
            BLOCK_SIZE_M=BLOCK_SIZE_M,
            BLOCK_SIZE_N=BLOCK_SIZE_N,
            DTYPE_FLAG=get_dtype_flag(embeddings.dtype)
        )
        output = output.contiguous()
        return cp.from_dlpack(output.reshape(*original_indices_shape, -1))

    else:

        ### Get the original shape ###
        original_indices_shape = indices.shape

        ### Flatten ###
        indices = indices.reshape(-1)

        ### Check Contiguous ###
        if not embeddings.flags.c_contiguous:
            embeddings = cp.ascontiguousarray(embeddings)
        if not indices.flags.c_contiguous:
            indices = cp.ascontiguousarray(indices)

        ### Create an empty output to copy embeds into ###
        with cp.cuda.Device(indices.device.id):
            output = cp.empty(
                (indices.shape[0], embeddings.shape[1]),
                dtype=embeddings.dtype
            )

        ### Get our dimensions ###
        n_elements = indices.shape[0]
        embed_dim = embeddings.shape[1]

        ### Vocab size is typically much larger than the embed_dim so lets set our ###
        ### block size based on the embed dim! ###
        BLOCK_SIZE_M = triton.next_power_of_2(min(128, embed_dim))
        BLOCK_SIZE_N = triton.next_power_of_2(min(128, embed_dim))

        grid = (
            triton.cdiv(n_elements, BLOCK_SIZE_M),
            triton.cdiv(embed_dim, BLOCK_SIZE_N)
        )

        embedding_forward_kernel[grid](
            embeddings_ptr=embeddings.data.ptr,
            indices_ptr=indices.data.ptr,
            output_ptr=output.data.ptr,
            n_elements=n_elements,
            embedding_dim=embed_dim,
            BLOCK_SIZE_M=BLOCK_SIZE_M,
            BLOCK_SIZE_N=BLOCK_SIZE_N,
            DTYPE_FLAG=get_dtype_flag(embeddings.dtype)
        )

        embeds = output.reshape(*original_indices_shape, -1)

        return embeds
```



```
    return embeds

def fused_embedding_backward(grad_output, embeddings, indices, use_dlpack=True):

    ### tl.atomic_add is extremely slow for bfloat16, so accumulate in float32 and cast back ###
    if str(embeddings.dtype) == "bfloat16":
        grad_weight = fused_embedding_backward(grad_output.astype(cp.float32),
                                               cp.empty(embeddings.shape, dtype=cp.float32),
                                               indices, use_dlpack=use_dlpack)
        return grad_weight.astype("bfloat16")

    ### Only support int32 indexes ###
    indices = indices.astype("int32", copy=False)

    ### Bounds checks force a GPU sync every call, so they are opt in ###
    if CHECK_INDICES:
        assert (indices >= 0).all(), f"Negative indices found: {indices.min()}"
        assert (indices < embeddings.shape[0]).all(), f"Index out of bounds: max={indices.max()}, vocab={embeddings.shape[0]}"

    if use_dlpack:

        ### Flatten ###
        indices = indices.reshape(-1)

        grad_output = torch.utils.dlpack.from_dlpack(grad_output)
        embeddings = torch.utils.dlpack.from_dlpack(embeddings)
        indices = torch.utils.dlpack.from_dlpack(indices)

        if not grad_output.is_contiguous():
            grad_output = grad_output.contiguous()
        if not embeddings.is_contiguous():
            embeddings = embeddings.contiguous()
        if not indices.is_contiguous():
            indices = indices.contiguous()

        grad_weight = torch.zeros_like(embeddings)

        n_elements = indices.shape[0]
        embed_dim = embeddings.shape[1]

        BLOCK_SIZE_M = triton.next_power_of_2(min(128, embed_dim))
        BLOCK_SIZE_N = triton.next_power_of_2(min(128, embed_dim))
        grid = (
            triton.cdiv(n_elements, BLOCK_SIZE_M),
            triton.cdiv(embed_dim, BLOCK_SIZE_N),
        )

        embedding_backward_kernel[grid](
            grad_output,
            grad_weight,
            indices,
            n_elements,
            embedding_dim=embed_dim,
            BLOCK_SIZE_M=BLOCK_SIZE_M,
            BLOCK_SIZE_N=BLOCK_SIZE_N,
            DTYPE_FLAG=get_dtype_flag(embeddings.dtype)
        )

        return cp.from_dlpack(grad_weight)

    else:

        ### Flatten ###
        indices = indices.reshape(-1)

        if not grad_output.flags.c_contiguous:
            grad_output = cp.ascontiguousarray(grad_output)
        if not embeddings.flags.c_contiguous:
            embeddings = cp.ascontiguousarray(embeddings)
        if not indices.flags.c_contiguous:
            indices = cp.ascontiguousarray(indices)

        grad_weight = cp.zeros_like(embeddings)

        n_elements = indices.shape[0]
        embed_dim = embeddings.shape[1]

        BLOCK_SIZE_M = triton.next_power_of_2(min(128, embed_dim))
        BLOCK_SIZE_N = triton.next_power_of_2(min(128, embed_dim))

        grid = (
            triton.cdiv(n_elements, BLOCK_SIZE_M),
            triton.cdiv(embed_dim, BLOCK_SIZE_N),
        )

        embedding_backward_kernel[grid](
            grad_output.data.ptr,
            grad_weight.data.ptr,
            indices.data.ptr,
            n_elements,
            embedding_dim=embed_dim,
            BLOCK_SIZE_M=BLOCK_SIZE_M,
            BLOCK_SIZE_N=BLOCK_SIZE_N,
            DTYPE_FLAG=get_dtype_flag(embeddings.dtype)
        )

        return grad_weight
```

